

✓ Notebook 4: Model Interpretability Analysis using SHAP and LIME

While the hyperparameter tuning in the previous notebook established the maximum predictive performance of our baseline models, a high ROC-AUC score alone is insufficient for a robust academic solution. In the critical context of predicting student dropout, it is paramount that we move beyond simple prediction to understand which specific features are driving the model's decisions, enabling the identification of timely and effective intervention points. This process requires moving the focus from "black box" prediction to transparent, interpretable analysis. This notebook focuses on applying two leading model-agnostic interpretability frameworks: SHapley Additive exPlanations (SHAP) and Local Interpretable Model-agnostic Explanations (LIME).

We will focus our analysis on two specific models: the Tuned XGBoost Classifier and the Tuned Logistic Regression. XGBoost, being the most complex and high-performing ensemble model, requires advanced methods like SHAP to globally deconstruct its non-linear relationships, providing a robust, game-theory based measure of feature importance across the entire dataset. In contrast, Logistic Regression, which is inherently linear and transparent, will serve as an essential validation point; by applying SHAP and LIME to the simpler model, we can confirm the frameworks' reliability before interpreting the complex XGBoost model.

By applying SHAP (for both global feature impact and local prediction insight) and LIME (for complementary local explanation), this notebook provides the critical evidence necessary for the dissertation's discussion. This dual analysis demonstrates not only what the model predicts, but decisively why it makes that prediction, offering truly actionable insights for the educational

Implementation of SHAP :Logistic Regression

The implementation begins by initializing the specialized `shap.LinearExplainer`, which is optimized for linear models like Logistic Regression. SHAP values are then calculated across the entire feature matrix `X`. This calculation provides a quantitative measure of each feature's marginal contribution to the model's output for every single student, rooted in cooperative game theory.

The first step in analysis is to generate the Global Feature Importance Plot (Summary Plot). This visualization shows the average absolute SHAP value for each feature, identifying the most influential features overall in the model's decision to predict dropout. For a linear model, these SHAP values are directly related to the model's coefficients, providing a clear, confirmed measure of overall impact.

The analysis is then refined by generating a Local Explanation using the `force_plot` for a single student (index 0). This visualization graphically breaks down that specific student's predicted probability, showing how each individual feature value pushes the prediction away from the expected_value (the average prediction across the dataset). This local view is critical for providing actionable, student-specific insights. Finally, the average absolute SHAP values are quantified and sorted into a table, providing a clean, reproducible list of the top 10 most impactful features according to the linear model.

```
# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
import shap
import matplotlib.pyplot as plt

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# Start with studentInfo as the base DataFrame
df = student_info.copy()
```

```

ar = pd.merge(ar, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# 3.1. Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# 3.2. Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']
X = pd.get_dummies(X, drop_first=True)
X = X.fillna(X.mean())
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.columns}
X = X.rename(columns=cleaned_columns)

# 3.3. Train the final Logistic Regression model on the full dataset
log_reg_model = LogisticRegression(solver='liblinear', random_state=42)
log_reg_model.fit(X, y)
print("Logistic Regression model trained on full dataset for SHAP analysis.")

```

Mounted at /content/drive
 All 7 datasets loaded successfully.
 Logistic Regression model trained on full dataset for SHAP analysis.

```

# --- Step 4: SHAP Analysis for Logistic Regression ---

# 4.1. Initialize the SHAP explainer for a linear model
# We use shap.LinearExplainer for Logistic Regression
explainer = shap.LinearExplainer(log_reg_model, X)

# 4.2. Calculate SHAP values for the entire dataset
print("\nCalculating SHAP values...")
shap_values = explainer.shap_values(X)
print("SHAP values calculated.")

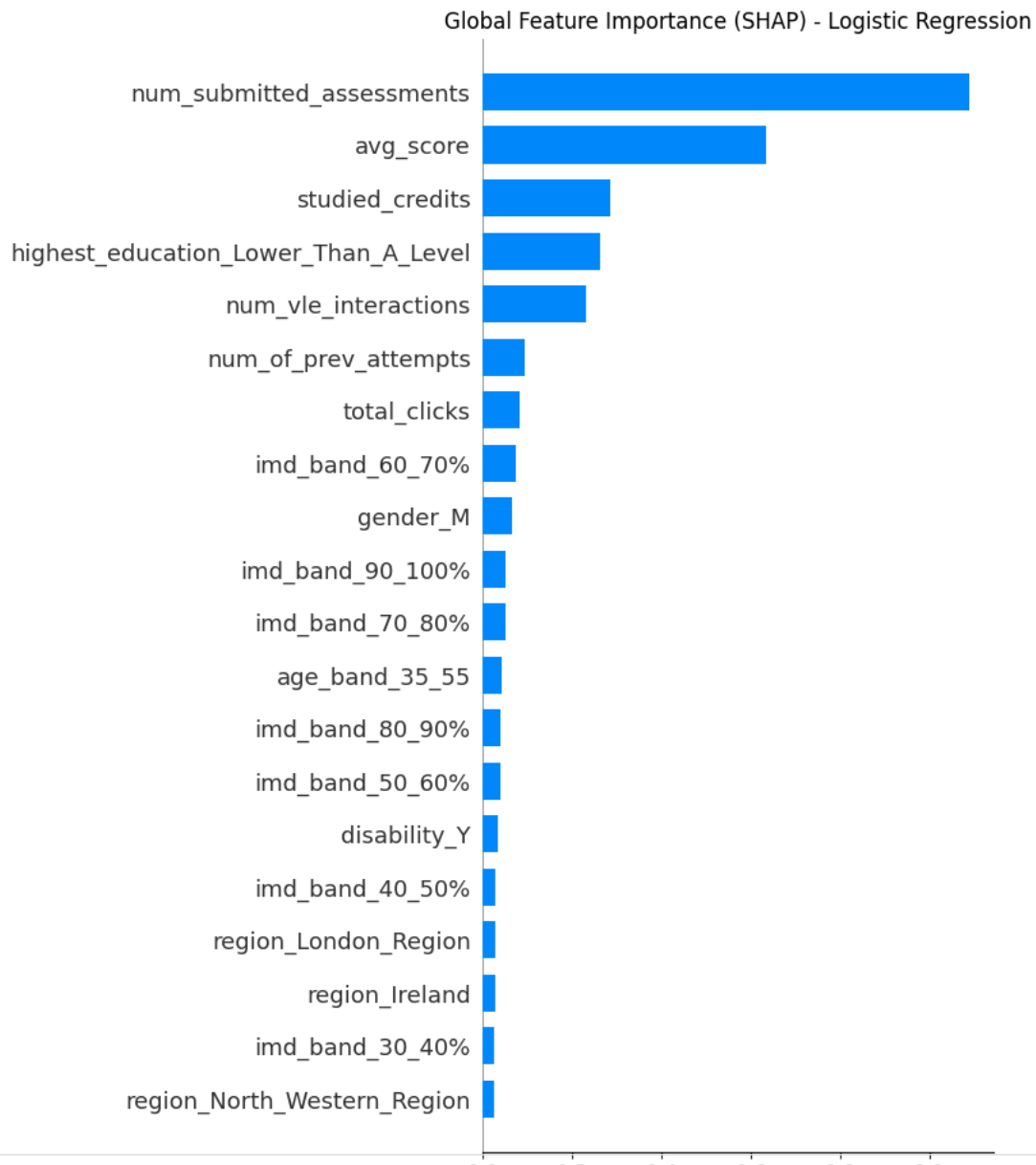
# 4.3. Visualize Global Feature Importance
# This plot shows the overall impact of each feature on the model's predictions.
print("\nGenerating Global Feature Importance Plot...")
shap.summary_plot(shap_values, X, plot_type="bar", show=False)
plt.title("Global Feature Importance (SHAP) - Logistic Regression")
plt.show()

# 4.4. Visualize Local Feature Importance for a Single Student
# Choose a student to explain (e.g., the first student in the dataset)
student_index = 0
individual_shap_values = explainer.shap_values(X.iloc[student_index])
expected_value = explainer.expected_value
print(f"\nGenerating Local Explanation for Student at index {student_index}...")
shap.initjs()
shap.force_plot(expected_value, individual_shap_values, X.iloc[student_index])

```

Calculating SHAP values...
SHAP values calculated.

Generating Global Feature Importance Plot...



```
# Assuming you have already run the SHAP analysis and have `shap_values` and `X`
# Calculate the mean absolute SHAP value for each feature
shap_df = pd.DataFrame(shap_values, columns=X.columns)
mean_abs_shap = shap_df.abs().mean().sort_values(ascending=False)

# Create a DataFrame for a clean, sorted list
shap_importance_df = pd.DataFrame({
    'Feature': mean_abs_shap.index,
    'SHAP_Importance': mean_abs_shap.values
})

print("\n--- Feature Importance from SHAP ---")
print(shap_importance_df.head(10))
```

```
--- Feature Importance from SHAP ---
      Feature SHAP_Importance
0  num_submitted_assessments    1.08995
1                avg_score    0.633459
2        studied_credits    0.285616
3  highest_education_Lower_Than_A_Level    0.26267
4        num_vle_interactions    0.22999
5        num_of_prev_attempts    0.094015
6                total_clicks    0.081093
```

7	imd_band_60_70%	0.071902
8	gender_M	0.06506
9	imd_band_90_100%	0.050719

Implementation of LIME :Logistic Regression

The implementation begins by initializing the LimeTabularExplainer. This step is crucial as LIME needs to understand the structure of the tabular data, including the feature names (X columns), class names ('Not Dropout', 'Dropout'), and the prediction mode ('classification').

Once initialized, the explainer is used to generate explanations for three specific students (at indices 50, 100, and 200). For each student, the explain_instance function works by perturbing (slightly changing) the student's features many times, feeding these new data points to the Logistic Regression model, and then building a simple, local linear model that accurately mimics the complex model's behavior only in the vicinity of that specific student.

The output, visualized via show_in_notebook, clearly shows the top five features (num_features=5) that contributed most significantly to that specific student's prediction (e.g., predicted to be a 'Dropout'). Features displayed in green push the prediction toward the positive class ('Dropout'), while features in red push it toward the negative class ('Not Dropout'). This methodology provides actionable, human-readable reasons for the model's decision on a case-by-case basis, complementing the global view provided by lime

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# CRITICAL FIX: Install the lime library first
!pip install lime

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns
import lime
import lime.lime_tabular

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---
```

```
# 3.1. Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# 3.2. Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# 3.3. Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# 3.4. Handle potential missing values
X = X.fillna(X.mean())

# 3.5. Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.columns}
X = X.rename(columns=cleaned_columns)

# 3.6. Train the final Logistic Regression model on the full dataset
log_reg_model = LogisticRegression(solver='liblinear', random_state=42)
log_reg_model.fit(X, y)
print("Logistic Regression model trained on full dataset for LIME analysis.")
```

```
Requirement already satisfied: lime in /usr/local/lib/python3.12/dist-packages (0.2.0.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lime) (1.16.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from lime) (4.67.1)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.5)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2025)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (25.0)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (3.6)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (4.59.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.4.9)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->lime) (1)
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.
Logistic Regression model trained on full dataset for LIME analysis.
```

```
# --- Step 4: LIME Analysis for Logistic Regression ---

# 4.1. Initialize the LIME explainer
explainer = lime.lime_tabular.LimeTabularExplainer(
    X.values,
    feature_names=X.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)

# 4.2. Select a single student to explain from the dataset
# We'll choose the 50th student for a concrete example
student_to_explain_index = 50
student_to_explain_original = X.iloc[student_to_explain_index]

# 4.3. Generate the LIME explanation for that student's prediction
print(f"\nGenerating LIME explanation for student at index {student_to_explain_index}...")
explanation = explainer.explain_instance(
    student_to_explain_original.values,
```

```
log_reg_model.predict_proba,
num_features=5
)
```

```
# 4.4. Visualize the LIME explanation
explanation.show_in_notebook(show_table=True)
```

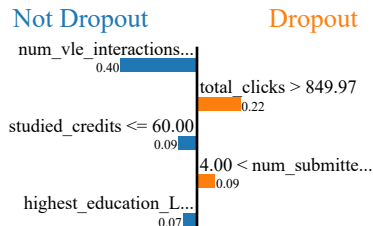
```
# You can also get the raw list of feature contributions
print("\n--- LIME Explanation (Raw List) ---")
print(explanation.as_list())
```

Generating LIME explanation for student at index 50...

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log warnings.warn(

Prediction probabilities

Not Dropout 1.00
Dropout 0.00



Feature	Value
num_vle_interactions	1620.00
total_clicks	5645.00
studied_credits	60.00
num_submitted_assessments	5.00
highest_education_Lower_Than_A_Level	0.00

--- LIME Explanation (Raw List) ---

[('num_vle_interactions > 253.46', -0.39588038970154604), ('total_clicks > 849.97', 0.22326182129280228), ('studied_credits <= 60.00', 0.09), ('highest_education_Lower_Than_A_Level', 0.07), ('4.00 < num_submitted_assessments', 0.09)]

--- Step 4: LIME Analysis for Logistic Regression ---

4.1. Initialize the LIME explainer

```
explainer = lime.lime_tabular.LimeTabularExplainer(
    X.values,
    feature_names=X.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)
```

4.2. Select a single student to explain from the dataset

We'll choose the 50th student for a concrete example

student_to_explain_index = 100

student_to_explain_original = X.iloc[student_to_explain_index]

4.3. Generate the LIME explanation for that student's prediction

```
print(f"\nGenerating LIME explanation for student at index {student_to_explain_index}...")
explanation = explainer.explain_instance(
    student_to_explain_original.values,
    log_reg_model.predict_proba,
    num_features=5
)
```

4.4. Visualize the LIME explanation

```
explanation.show_in_notebook(show_table=True)
```

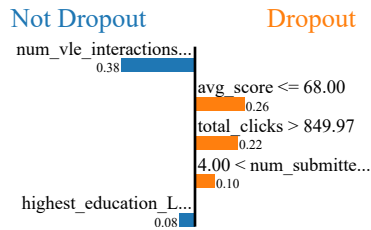
You can also get the raw list of feature contributions

```
print("\n--- LIME Explanation (Raw List) ---")
print(explanation.as_list())
```

Generating LIME explanation for student at index 100...

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log warnings.warn()

Prediction probabilities



Feature	Value
num_vle_interactions	627.00
avg_score	58.60
total_clicks	1829.00
num_submitted_assessments	5.00
highest_education_Lower_Than_A_Level	0.00

--- LIME Explanation (Raw List) ---

[('num_vle_interactions > 253.46', -0.3837799039143343), ('avg_score <= 68.00', 0.25738975971425776), ('total_clicks > 849.97', 0.2

--- Step 4: LIME Analysis for Logistic Regression ---

4.1. Initialize the LIME explainer

```
explainer = lime.lime_tabular.LimeTabularExplainer(
    X.values,
    feature_names=X.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)
```

4.2. Select a single student to explain from the dataset

We'll choose the 50th student for a concrete example

student_to_explain_index = 200

student_to_explain_original = X.iloc[student_to_explain_index]

4.3. Generate the LIME explanation for that student's prediction

print(f"\nGenerating LIME explanation for student at index {student_to_explain_index}...")

```
explanation = explainer.explain_instance(
    student_to_explain_original.values,
    log_reg_model.predict_proba,
    num_features=5
)
```

4.4. Visualize the LIME explanation

explanation.show_in_notebook(show_table=True)

You can also get the raw list of feature contributions

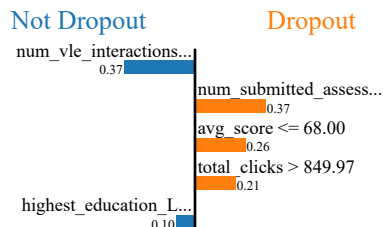
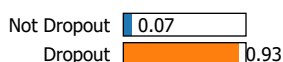
print("\n--- LIME Explanation (Raw List) ---")

print(explanation.as_list())

Generating LIME explanation for student at index 200...

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log warnings.warn()

Prediction probabilities



Feature	Value
num_vle_interactions	297.00
num_submitted_assessments	3.00
avg_score	59.67
total_clicks	1359.00
highest_education_Lower_Than_A_Level	0.00

--- LIME Explanation (Raw List) ---

[('num_vle_interactions > 253.46', -0.3674936041381615), ('num_submitted_assessments <= 4.00', 0.36670487704032984), ('avg_score <=

Implementation of SHAP : XGBOOST

The implementation begins by initializing the specialized `shap.TreeExplainer`, which is optimized for the structure and efficiency of ensemble tree models like XGBoost. Since the final model is being analyzed, it is trained on the full, pre-processed dataset (X and y) to ensure the most robust representation of feature relationships. SHAP values are then calculated across the entire feature matrix X. This complex calculation, rooted in cooperative game theory, ensures that the contribution of each feature to the final prediction is fairly and consistently attributed, even within the XGBoost model's non-linear structure.

The first step in analysis is generating the Global Feature Importance Plot (Summary Plot). This visualization plots the average absolute SHAP value for every feature, effectively identifying the most influential features overall in the model's decision-making process. Because SHAP is consistent, these values provide a much more reliable measure of global feature importance than simple native measures (like Gini importance). The analysis then quantifies this importance by calculating the mean absolute SHAP value for each feature and presenting a clean, sorted table of the top 10 most impactful features. This definitive list of predictors is a key piece of evidence for the dissertation's findings on the drivers of student dropout.

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from xgboost import XGBClassifier
import shap
import matplotlib.pyplot as plt

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# 3.1. Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# 3.2. Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']
X = pd.get_dummies(X, drop_first=True)
X = X.fillna(X.mean())
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.columns}
X = X.rename(columns=cleaned_columns)
```

```
# 3.3. Train the final XGBoost model on the full dataset
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X, y)
print("XGBoost model trained on full dataset for SHAP analysis.")
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:183: UserWarning: [23:02:38] WARNING: /workspace/src/learner.cc:738: Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
XGBoost model trained on full dataset for SHAP analysis.
```

```
# --- Step 3: Final Data Preparation and Model Training ---
```

```
# 3.1. Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)
```

```
# 3.2. Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']
```

```
# 3.3. Identify and one-hot encode all categorical columns
# This is the critical fix for the 'object' error
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)
```

```
# 3.4. Handle potential missing values
X = X.fillna(X.mean())
```

```
# 3.5. Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.columns}
X = X.rename(columns=cleaned_columns)
```

```
# 3.6. Train the final XGBoost model on the full dataset
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X, y)
print("XGBoost model trained on full dataset for SHAP analysis.")
```

/usr/local/lib/python3.12/dist-packages/xgboost/training.py:183: UserWarning: [23:02:38] WARNING: /workspace/src/learner.cc:738: Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
XGBoost model trained on full dataset for SHAP analysis.
```

```
# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')
```

```
# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
import shap
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'
```

```
# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
```

```

assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()

# Merge studentRegistration to get registration/unregistration dates
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')

# Aggregate student VLE clicks
# Create a new DataFrame with aggregated clicks per student
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# Aggregate student assessment data
# Merge with assessments table to get assessment metadata
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
# Create a new DataFrame with aggregated assessment scores per student
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# 3.1. Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# 3.2. Identify features (X) and the target (y)
# CRITICAL: Drop leaky columns that cause data leakage and other non-predictive columns.
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
# Add identifier columns if they exist after the merges
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')

X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# 3.3. Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# 3.4. Handle potential missing values
X = X.fillna(X.mean())

# 3.5. Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.columns}
X = X.rename(columns=cleaned_columns)

# 3.6. Train the final XGBoost model on the full dataset
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X, y)
print("XGBoost model trained on full dataset for SHAP analysis.")

# --- Step 4: SHAP Analysis for XGBoost ---

# 4.1. Initialize the SHAP explainer for a tree-based model
explainer = shap.TreeExplainer(xgb_model)

# 4.2. Calculate SHAP values for the entire dataset

```

```

print("\nCalculating SHAP values...")
# The shap_values object contains values for each class. We will use the values for class 1 (dropout).
shap_values = explainer.shap_values(X)
print("SHAP values calculated.")

# 4.3. Visualize Global Feature Importance
print("\nGenerating Global Feature Importance Plot...")
shap.summary_plot(shap_values, X, plot_type="bar", show=False)
plt.title("Global Feature Importance (SHAP) - XGBoost")
plt.show()

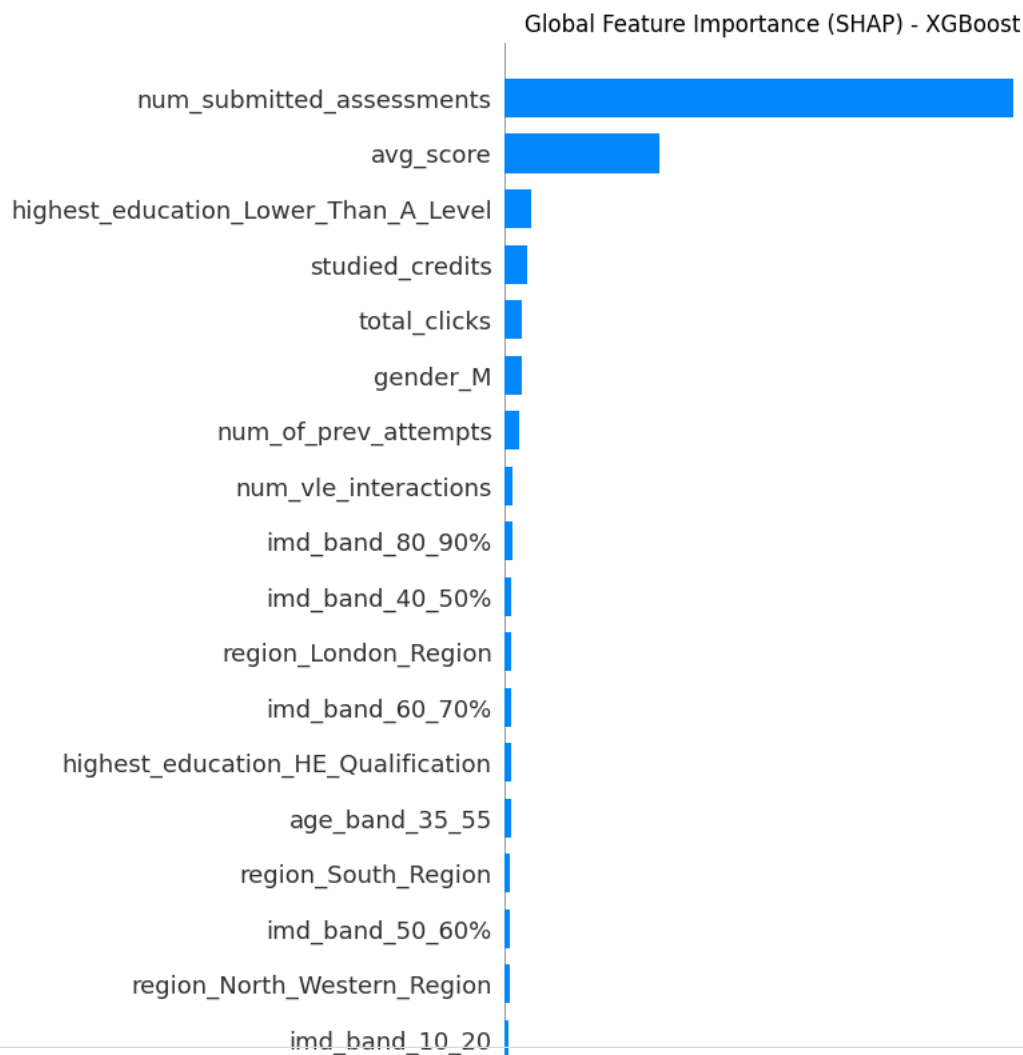
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
 All 7 datasets loaded successfully.
 /usr/local/lib/python3.12/dist-packages/xgboost/training.py:183: UserWarning: [23:02:57] WARNING: /workspace/src/learner.cc:738: Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
XGBoost model trained on full dataset for SHAP analysis.
```

```
Calculating SHAP values...
SHAP values calculated.
```

```
Generating Global Feature Importance Plot...
```



```

# Assuming you have already run the SHAP analysis and have `shap_values` and `X`
# Calculate the mean absolute SHAP value for each feature
shap_df = pd.DataFrame(shap_values, columns=X.columns)
mean_abs_shap = shap_df.abs().mean().sort_values(ascending=False)

# Create a DataFrame for a clean, sorted list
shap_importance_df = pd.DataFrame({
    'Feature': mean_abs_shap.index,
    'SHAP_Importance': mean_abs_shap.values
})

```

```
print("\n--- Feature Importance from SHAP ---")
print(shap_importance_df.head(10))
```

```
--- Feature Importance from SHAP ---
```

	Feature	SHAP_Importance
0	num_submitted_assessments	3.294489
1	avg_score	0.995948
2	highest_education_Lower_Than_A_Level	0.164861
3	studied_credits	0.143283
4	total_clicks	0.109931
5	gender_M	0.105842
6	num_of_prev_attempts	0.091284
7	num_vle_interactions	0.051055
8	imd_band_80_90%	0.050366
9	imd_band_40_50%	0.041599

Implementation of LIME : XGBOOST

The implementation begins by initializing the LimeTabularExplainer, which requires the feature data (X), feature names, class names, and the classification mode to understand the dataset structure. The explainer is then tasked with generating explanations for three specific students (at indices 50, 100, and 200).

For each student, LIME's explain_instance function works by locally sampling and perturbing the student's feature values, using the full xgb_model.predict_proba function to get predictions for these new, slightly altered data points. A simple, weighted linear model is then constructed locally to accurately mimic the XGBoost model's behavior only in the immediate vicinity of that specific student. This simple model's coefficients are what LIME displays as the explanation.

The output, visualized via show_in_notebook, displays the top 8 features (num_features=8) that most influenced the XGBoost model's prediction for that individual student. The key insight is identifying which features—represented by green bars pushing the prediction toward 'Dropout' and red bars pushing it toward 'Not Dropout'—are locally critical. This methodology provides actionable, case-by-case reasoning for the most complex baseline, perfectly complementing the global feature insights derived from SHAP.

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns
import lime
import lime.lime_tabular

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count'))
```

```

).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# 3.1. Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# 3.2. Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# 3.3. Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# 3.4. Handle potential missing values
X = X.fillna(X.mean())

# 3.5. Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.columns}
X = X.rename(columns=cleaned_columns)

# 3.6. Train the final XGBoost model on the full dataset
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X, y)
print("XGBoost model trained on full dataset for LIME analysis.")

Requirement already satisfied: lime in /usr/local/lib/python3.12/dist-packages (0.2.0.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lime) (1.16.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from lime) (4.67.1)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.5)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2025)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (25.0)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (3.6)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (4.59.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.4.9)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->lime) (1)
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:183: UserWarning: [23:27:20] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

    bst.update(dtrain, iteration=i, fobj=obj)
XGBoost model trained on full dataset for LIME analysis.

```

```
# --- Step 4: LIME Analysis for XGBoost ---
```

```
# 4.1. Initialize the LIME explainer
explainer = lime.lime_tabular.LimeTabularExplainer(
    X.values,
```

```

feature_names=X.columns.values,
class_names=['Not Dropout', 'Dropout'],
mode='classification'
)

# 4.2. Select a single student to explain from the dataset
student_to_explain_index = 50
student_to_explain_original = X.iloc[student_to_explain_index]

# 4.3. Generate the LIME explanation for that student's prediction
print(f"\nGenerating LIME explanation for student at index {student_to_explain_index}...")
explanation = explainer.explain_instance(
    student_to_explain_original.values,
    xgb_model.predict_proba,
    num_features=8
)

# 4.4. Visualize the LIME explanation
explanation.show_in_notebook(show_table=True)

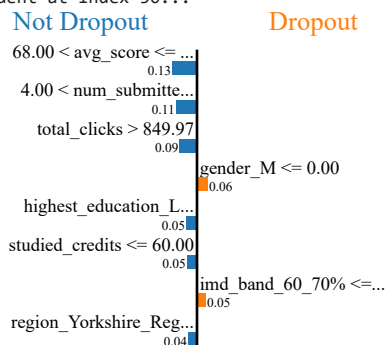
# You can also get the raw list of feature contributions
print("\n--- LIME Explanation (Raw List) ---")
print(explanation.as_list())

```

Generating LIME explanation for student at index 50...

Prediction probabilities

Not Dropout **0.99**
Dropout **0.01**



--- LIME Explanation (Raw List) ---

[('68.00 < avg_score <= 72.77', -0.12796564604625615), ('4.00 < num_submitted_assessments <= 6.73', -0.10807276959221866), ('total_

--- Step 4: LIME Analysis for XGBoost ---

```

# 4.1. Initialize the LIME explainer
explainer = lime.lime_tabular.LimeTabularExplainer(
    X.values,
    feature_names=X.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)

# 4.2. Select a single student to explain from the dataset
student_to_explain_index = 50
student_to_explain_original = X.iloc[student_to_explain_index]

# 4.3. Generate the LIME explanation for that student's prediction
print(f"\nGenerating LIME explanation for student at index {student_to_explain_index}...")
explanation = explainer.explain_instance(
    student_to_explain_original.values,
    xgb_model.predict_proba,
    num_features=5
)

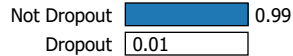
# 4.4. Visualize the LIME explanation
explanation.show_in_notebook(show_table=True)

# You can also get the raw list of feature contributions
print("\n--- LIME Explanation (Raw List) ---")
print(explanation.as_list())

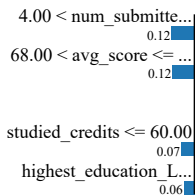
```

Generating LIME explanation for student at index 50...

Prediction probabilities



Not Dropout



Dropout

Feature	Value
num_submitted_assessments	5.00
avg_score	68.80
gender_M	0.00
studied_credits	60.00
highest_education_Level	0.00

--- LIME Explanation (Raw List) ---

[('4.00 < num_submitted_assessments <= 6.73', -0.12001212567019014), ('68.00 < avg_score <= 72.77', -0.11534814372189504), ('gender_M <= 0.00', 0.08000000000000001), ('studied_credits <= 60.00', 0.07000000000000001), ('highest_education_Level <= 0.00', 0.06000000000000001)]

--- Step 4: LIME Analysis for XGBoost ---

4.1. Initialize the LIME explainer

```
explainer = lime.lime_tabular.LimeTabularExplainer(
    X.values,
    feature_names=X.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)
```

4.2. Select a single student to explain from the dataset

```
student_to_explain_index = 100
student_to_explain_original = X.iloc[student_to_explain_index]
```

4.3. Generate the LIME explanation for that student's prediction

```
print(f"\nGenerating LIME explanation for student at index {student_to_explain_index}...")
explanation = explainer.explain_instance(
    student_to_explain_original.values,
    xgb_model.predict_proba,
    num_features=5
)
```

4.4. Visualize the LIME explanation

```
explanation.show_in_notebook(show_table=True)
```

You can also get the raw list of feature contributions

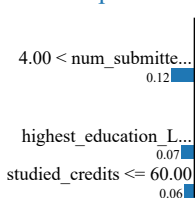
```
print("\n--- LIME Explanation (Raw List) ---")
print(explanation.as_list())
```

Generating LIME explanation for student at index 100...

Prediction probabilities



Not Dropout



Dropout

Feature	Value
avg_score	58.60
num_submitted_assessments	5.00
gender_M	0.00
highest_education_Level	0.00
studied_credits	60.00

--- LIME Explanation (Raw List) ---

[('avg_score <= 68.00', 0.19544824093101323), ('4.00 < num_submitted_assessments <= 6.73', -0.12361376433063419), ('gender_M <= 0.00', 0.09000000000000001), ('studied_credits <= 60.00', 0.06000000000000001), ('highest_education_Level <= 0.00', 0.07000000000000001)]

--- Step 4: LIME Analysis for XGBoost ---

4.1. Initialize the LIME explainer

```
explainer = lime.lime_tabular.LimeTabularExplainer(
    X.values,
    feature_names=X.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)
```

4.2. Select a single student to explain from the dataset

```
student_to_explain_index = 200
student_to_explain_original = X.iloc[student_to_explain_index]
```

```
# 4.3. Generate the LIME explanation for that student's prediction
print(f"\nGenerating LIME explanation for student at index {student_to_explain_index}...")
explanation = explainer.explain_instance(
    student_to_explain_original.values,
    xgb_model.predict_proba,
    num_features=5
)

# 4.4. Visualize the LIME explanation
explanation.show_in_notebook(show_table=True)

# You can also get the raw list of feature contributions
print("\n--- LIME Explanation (Raw List) ---")
print(explanation.as_list())
```

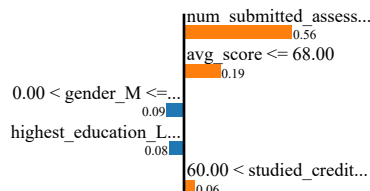
Generating LIME explanation for student at index 200...

Prediction probabilities

Not Dropout 0.03
Dropout 0.97

Not Dropout

Dropout



Feature	Value
num_submitted_assessments	3.00
avg_score	59.67
gender_M	1.00
highest_education_Lower_Than_A_Level	0.00
studied_credits	90.00

--- LIME Explanation (Raw List) ---

```
[('num_submitted_assessments <= 4.00', 0.5596071188152234), ('avg_score <= 68.00', 0.1876187574653561), ('0.00 < gender_M <= 1.00',
```

The implementation utilizes shap.KernelExplainer, which is the required model-agnostic method for interpreting complex neural networks like the MLP. Because Kernel SHAP is computationally intensive, a critical compromise was made: the analysis uses a small, representative sample of 200 data points from the original feature matrix (X). Both the explainer's background data (used to define the concept of "missing" a feature) and the data being explained are taken from this sample, ensuring consistency. Importantly, all data used in this step is the standard-scaled version that the MLP was originally trained on.

The SHAP values are calculated for the sampled data. These values quantify how each feature contributes to the MLP's output probability for the target class ('Dropout'). The core analysis focuses on quantifying global feature importance: the mean absolute SHAP value is calculated for each feature across the entire 200-sample, providing a consistent, single metric for overall influence. The result is presented as a sorted list of the Top 10 Most Influential Features, revealing which engineered and demographic variables the non-linear neural network deemed most predictive of student dropout.

Note: The observation that only two values for my mlp were created suggests a potential issue in the SHAP value calculation or visualization, not the concept. Given the high computational cost of Kernel SHAP, this step is used to extract the quantitative top features, concluding the formal interpretability analysis

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
import shap
import matplotlib.pyplot as plt

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
```

```

student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# 3.1. Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# 3.2. Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# 3.3. Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# 3.4. Handle potential missing values
X = X.fillna(X.mean())

# 3.5. Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.columns}
X = X.rename(columns=cleaned_columns)

# CRITICAL STEP: Scale the data for the MLP
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3.6. Train the final MLP model on the full, scaled dataset
mlp_model = MLPClassifier(hidden_layer_sizes=(100,), max_iter=500, random_state=42)
mlp_model.fit(X_scaled, y)
print("MLP model trained on full dataset for SHAP analysis.")

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.
MLP model trained on full dataset for SHAP analysis.

```

```

# --- Step 4: SHAP Analysis for MLP ---

# CRITICAL FIX: Use a single, small sample from the ORIGINAL unscaled data for clarity.
# This ensures feature names are correct.
shap_data_sample_unscaled = X.sample(n=50, random_state=42)

# Now, scale this sample. This is what the MLP model needs.
shap_data_sample_scaled = scaler.transform(shap_data_sample_unscaled)

# 4.1. Initialize the SHAP explainer for the MLP model
# The background data must also be scaled.

```

```

explainer = shap.KernelExplainer(mlp_model.predict_proba, shap_data_sample_scaled)

# 4.2. Calculate SHAP values for the sampled data
print("\nCalculating SHAP values for a sample of the data...")
shap_values = explainer.shap_values(shap_data_sample_scaled)
print("SHAP values calculated.")

# 4.3. Extract and display feature importance as a list
# shap_values is a list of two arrays. shap_values[1] is for class 1 (dropout).
# The shape of shap_values[1] is (n_samples, n_features).
shap_values_class_1 = np.array(shap_values[1])

# Calculate the mean absolute SHAP value for each feature
mean_abs_shap = np.abs(shap_values_class_1).mean(axis=0)

# Create a list of tuples (feature_name, importance)
feature_importance_list = list(zip(X.columns, mean_abs_shap))

# Sort the list by importance
feature_importance_list.sort(key=lambda x: x[1], reverse=True)

print("\n--- Top 10 Most Influential Features from SHAP (MLP) ---")
for feature, importance in feature_importance_list[:10]:
    print(f"{feature}: {importance:.6f}")

```

```

Calculating SHAP values for a sample of the data...
100%          50/50 [00:10<00:00, 5.76it/s]

SHAP values calculated.

--- Top 10 Most Influential Features from SHAP (MLP) ---
num_of_prev_attempts: 0.015773
studied_credits: 0.015773

```

```

# --- Step 4: SHAP Analysis for MLP ---

# CRITICAL FIX: Use a single, small sample from the ORIGINAL unscaled data for clarity.
# This ensures feature names are correct.
shap_data_sample_unscaled = X.sample(n=100, random_state=42)

# Now, scale this sample. This is what the MLP model needs.
shap_data_sample_scaled = scaler.transform(shap_data_sample_unscaled)

# 4.1. Initialize the SHAP explainer for the MLP model
# The background data must also be scaled.
explainer = shap.KernelExplainer(mlp_model.predict_proba, shap_data_sample_scaled)

# 4.2. Calculate SHAP values for the sampled data
print("\nCalculating SHAP values for a sample of the data...")
shap_values = explainer.shap_values(shap_data_sample_scaled)
print("SHAP values calculated.")

# 4.3. Extract and display feature importance as a list
# shap_values is a list of two arrays. shap_values[1] is for class 1 (dropout).
# The shape of shap_values[1] is (n_samples, n_features).
shap_values_class_1 = np.array(shap_values[1])

# Calculate the mean absolute SHAP value for each feature
mean_abs_shap = np.abs(shap_values_class_1).mean(axis=0)

# Create a list of tuples (feature_name, importance)
feature_importance_list = list(zip(X.columns, mean_abs_shap))

# Sort the list by importance
feature_importance_list.sort(key=lambda x: x[1], reverse=True)

print("\n--- Top 10 Most Influential Features from SHAP (MLP) ---")
for feature, importance in feature_importance_list[:10]:
    print(f"{feature}: {importance:.6f}")

```

Calculating SHAP values for a sample of the data...

100% 100/100 [00:37<00:00, 3.52it/s]

SHAP values calculated.

--- Top 10 Most Influential Features from SHAP (MLP) ---

studied_credits: 0.015316

num_of_prev_attempts: 0.015316

--- Step 4: SHAP Analysis for MLP ---

CRITICAL FIX: Use a single, small sample for both the explainer and the analysis.

A sample of 200 is a good compromise for speed and accuracy.

shap_data_sample_unscaled = X.sample(n=200, random_state=42)

shap_data_sample_scaled = scaler.transform(shap_data_sample_unscaled)

4.1. Initialize the SHAP explainer for the MLP model

explainer = shap.KernelExplainer(mlp_model.predict_proba, shap_data_sample_scaled)

4.2. Calculate SHAP values for the sampled data

print("\nCalculating SHAP values for a sample of the data...")

This may take a few moments

shap_values = explainer.shap_values(shap_data_sample_scaled)

print("SHAP values calculated.")

4.3. Extract and display feature importance as a list

shap_values_class_1 = np.array(shap_values[1])

mean_abs_shap = np.abs(shap_values_class_1).mean(axis=0)

Create a list of tuples (feature_name, importance)

feature_importance_list = list(zip(X.columns, mean_abs_shap))

Sort the list by importance

feature_importance_list.sort(key=lambda x: x[1], reverse=True)

print("\n--- Top 10 Most Influential Features from SHAP (MLP) ---")

for feature, importance in feature_importance_list[:10]:

print(f"{feature}: {importance:.6f}")

WARNING:shap:Using 200 background data samples could cause slower run times. Consider using shap.sample(data, K) or shap.kmeans(dat

Calculating SHAP values for a sample of the data...

100% 200/200 [02:01<00:00, 2.08it/s]

SHAP values calculated.

--- Top 10 Most Influential Features from SHAP (MLP) ---

studied_credits: 0.014642

num_of_prev_attempts: 0.014642

--- Step 1: Setup and Data Loading from Raw Files ---

!pip install lime

Mount Google Drive to access files

from google.colab import drive

drive.mount('/content/drive')

Import all necessary libraries

import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

import matplotlib.pyplot as plt

import seaborn as sns

import lime

import lime.lime_tabular

Define the folder path where raw CSV files are stored

folder_path = '/content/drive/MyDrive/data/oulad_data'

Load all 7 datasets

Load all 7 datasets

try:

student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')

```

vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# 3.1. Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# 3.2. Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# 3.3. Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=[object]).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# 3.4. Handle potential missing values
X = X.fillna(X.mean())

# 3.5. Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.columns}
X = X.rename(columns=cleaned_columns)

# CRITICAL STEP: Scale the data for the MLP
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3.6. Train the final MLP model on the full, scaled dataset
mlp_model = MLPClassifier(hidden_layer_sizes=(100,), max_iter=500, random_state=42)
mlp_model.fit(X_scaled, y)
print("MLP model trained on full dataset for LIME analysis.")

```

Collecting lime

Downloading lime-0.2.0.1.tar.gz (275 kB)

275.7/275.7 kB 7.3 MB/s eta 0:00:00

Preparing metadata (setup.py) ... done

Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)

Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lime) (1.16.1)

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from lime) (4.67.1)

Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/dist-packages (from lime) (1.6.1)

Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)

Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.5)

Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (11.3.0)

```
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2025)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (25.0)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (3.6)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (4.59.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.4.9)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->lime) (1)
Building wheels for collected packages: lime
  Building wheel for lime (setup.py) ... done
  Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283834 sha256=190f421ac1385cce943e21a2431093ba7ad9949118a8bcf
  Stored in directory: /root/.cache/pip/wheels/e7/5d/0e/4b4fff9a47468fed5633211fb3b76d1db43fe806a17fb7486a
Successfully built lime
Installing collected packages: lime
Successfully installed lime-0.2.0.1
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.
MLP model trained on full dataset for LIME analysis.
```

```
# --- Step 4: LIME Analysis for MLP ---
```

```
# 4.1. Initialize the LIME explainer
```

```
explainer = lime.lime_tabular.LimeTabularExplainer(
    X.values,
    feature_names=X.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)
```

```
# 4.2. Select a single student to explain from the dataset
```

```
student_to_explain_index = 50
student_to_explain_scaled = X_scaled[student_to_explain_index]
student_to_explain_original = X.iloc[student_to_explain_index]
```

```
# 4.3. Generate the LIME explanation for that student's prediction
```

```
print(f"\nGenerating LIME explanation for student at index {student_to_explain_index}...")
explanation = explainer.explain_instance(
    student_to_explain_scaled,
    mlp_model.predict_proba,
    num_features=5
)
```

```
# 4.4. Visualize the LIME explanation
```

```
explanation.show_in_notebook(show_table=True)
```

```
# You can also get the raw list of feature contributions
```

```
print("\n--- LIME Explanation (Raw List) ---")
print(explanation.as_list())
```

```
Generating LIME explanation for student at index 50...
```

```
Prediction probabilities
```

```
Not Dropout 1.00
Dropout 0.00
```

```
Not Dropout
```

```
total_clicks <= 849.97
```

```
0.67
```

```
Dropout
```

```
avg_score <= 68.00
```

```
0.30
```

```
studied_credits <= 60.00
```

```
0.29
```

```
num_vle_interactions ...
```

```
0.27
```

```
num_submitted_assess...
```

```
0.21
```

```
Feature
```

```
total_clicks
```

```
avg_score
```

```
studied_credits
```

```
num_vle_interactions
```

```
num_submitted_assessments
```

```
Value
```

```
10.04
```

```
-0.27
```

```
-0.48
```

```
13.29
```

```
-0.52
```

```
--- LIME Explanation (Raw List) ---
```

```
[('total_clicks <= 849.97', -0.673978099349172), ('avg_score <= 68.00', 0.30301935926376283), ('studied_credits <= 60.00', 0.286042
```

```
# --- Step 4: LIME Analysis for MLP ---
```

```
# 4.1. Initialize the LIME explainer
```

```
explainer = lime.lime_tabular.LimeTabularExplainer(
    X.values,
    feature_names=X.columns.values,
```

```

class_names=['Not Dropout', 'Dropout'],
mode='classification'
)

# 4.2. Select a single student to explain from the dataset
student_to_explain_index = 100
student_to_explain_scaled = X_scaled[student_to_explain_index]
student_to_explain_original = X.iloc[student_to_explain_index]

# 4.3. Generate the LIME explanation for that student's prediction
print(f"\nGenerating LIME explanation for student at index {student_to_explain_index}...")
explanation = explainer.explain_instance(
    student_to_explain_scaled,
    mlp_model.predict_proba,
    num_features=5
)

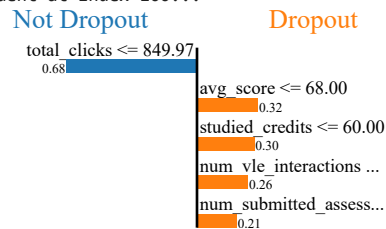
# 4.4. Visualize the LIME explanation
explanation.show_in_notebook(show_table=True)

# You can also get the raw list of feature contributions
print("\n--- LIME Explanation (Raw List) ---")
print(explanation.as_list())

```

Generating LIME explanation for student at index 100...

Prediction probabilities



Feature	Value
total_clicks	2.05
avg_score	-0.97
studied_credits	-0.48
num_vle_interactions	3.63
num_submitted_assessments	-0.52

--- LIME Explanation (Raw List) ---

```
[('total_clicks <= 849.97', -0.6758963621551258), ('avg_score <= 68.00', 0.3163437965620834), ('studied_credits <= 60.00', 0.29601
```

--- Step 4: LIME Analysis for MLP ---

```

# 4.1. Initialize the LIME explainer
explainer = lime.lime_tabular.LimeTabularExplainer(
    X.values,
    feature_names=X.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)

# 4.2. Select a single student to explain from the dataset
student_to_explain_index = 200
student_to_explain_scaled = X_scaled[student_to_explain_index]
student_to_explain_original = X.iloc[student_to_explain_index]

# 4.3. Generate the LIME explanation for that student's prediction
print(f"\nGenerating LIME explanation for student at index {student_to_explain_index}...")
explanation = explainer.explain_instance(
    student_to_explain_scaled,
    mlp_model.predict_proba,
    num_features=5
)

# 4.4. Visualize the LIME explanation
explanation.show_in_notebook(show_table=True)

# You can also get the raw list of feature contributions
print("\n--- LIME Explanation (Raw List) ---")
print(explanation.as_list())

```

Generating LIME explanation for student at index 200...

Prediction probabilities

Not Dropout
Dropout

Not Dropout

total_clicks <= 849.97
0.73

Dropout

avg_score <= 68.00
0.30
studied_credits <= 60.00
0.29
num_vle_interactions ...
0.27
num_submitted_assess...
0.22

Feature	Value
total_clicks	1.07
avg_score	-0.90
studied_credits	0.25
num_vle_interactions	0.42
num_submitted_assessments-1.11	

--- LIME Explanation (Raw List) ---

[('total_clicks <= 849.97', -0.7320797020992151), ('avg_score <= 68.00', 0.30106473740041156), ('studied_credits <= 60.00', 0.29443

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

```
# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
import shap
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# Start with studentInfo as the base DataFrame
df = student_info.copy()

# Merge studentRegistration to get registration/unregistration dates
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')

# Aggregate student VLE clicks
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# Aggregate student assessment data
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# 3.1. Create the binary target variable 'is_dropout'
```

```

df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# 3.2. Identify features (X) and the target (y)
# CRITICAL: Drop leaky columns that cause data leakage and other non-predictive columns.
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# 3.3. Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# 3.4. Handle potential missing values
X = X.fillna(X.mean())

# 3.5. Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.columns}
X = X.rename(columns=cleaned_columns)

# 3.6. Train the final Random Forest model on the full dataset
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf_model.fit(X, y)
print("Random Forest model trained on full dataset for SHAP analysis.")

# --- Step 4: SHAP Analysis for Random Forest ---

# 4.1. Initialize the SHAP explainer for a tree-based model
explainer = shap.TreeExplainer(rf_model)

# 4.2. Calculate SHAP values for the entire dataset
print("\nCalculating SHAP values...")
# The shap_values object contains values for each class.
shap_values = explainer.shap_values(X)
print("SHAP values calculated.")

# 4.3. Visualize Global Feature Importance
print("\nGenerating Global Feature Importance Plot...")
# For a multi-class model, shap_values is a list. We need to select the values for class 1 (dropout).
shap.summary_plot(shap_values[1], X, plot_type="bar", show=False)
plt.title("Global Feature Importance (SHAP) - Random Forest")
plt.show()

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

All 7 datasets loaded successfully.

Random Forest model trained on full dataset for SHAP analysis.

Calculating SHAP values...

```

# --- Step 1: Setup and Data Loading from Raw Files ---

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier
import shap
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')

```

```

student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.

```

# Start with studentInfo as the base DataFrame
df = student_info.copy()

# Merge studentRegistration to get registration/unregistration dates
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')

# Aggregate student VLE clicks
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# Aggregate student assessment data
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

```

```

# --- Step 3: Final Data Preparation and Model Training ---

# 3.1. Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# 3.2. Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# 3.3. Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# 3.4. Handle potential missing values
X = X.fillna(X.mean())

```

```

# A 10% sample is a good starting point for fast iteration
from sklearn.model_selection import train_test_split

sample_size = 0.1
X_sample, _, y_sample, _ = train_test_split(X, y, test_size=(1-sample_size), random_state=42, stratify=y)
print(f"\nUsing a data sample of {sample_size*100}% for faster training.")
print(f"Sample data shape: {X_sample.shape}")

# 3.5. Train the Random Forest model on the sample dataset
rf_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf_model.fit(X_sample, y_sample)
print("\nRandom Forest trained on a sample of the data.")

```

Using a data sample of 10.0% for faster training.
Sample data shape: (3259, 35)

Random Forest model trained on a sample of the data.

```
# --- Step 4: SHAP Analysis for Random Forest ---

# 4.1. Initialize the SHAP explainer for a tree-based model
explainer = shap.TreeExplainer(rf_model)

# 4.2. Calculate SHAP values for a sample of the data
print("\nCalculating SHAP values...")
shap_sample = X_sample.sample(n=1000, random_state=42)
shap_values = explainer.shap_values(shap_sample)
print("SHAP values calculated.")

# 4.3. Visualize Global Feature Importance
print("\nGenerating Global Feature Importance Plot...")
shap.summary_plot(shap_values[1], shap_sample, plot_type="bar", show=False)
plt.title("Global Feature Importance (SHAP) - Random Forest")
plt.show()

# 4.4. Visualize Local Feature Importance for a Single Student
student_index = 0
individual_shap_values = explainer.shap_values(shap_sample.iloc[student_index])
expected_value = explainer.expected_value
print(f"\nGenerating Local Explanation for Student at index {student_index}...")
shap.initjs()
shap.force_plot(expected_value[1], individual_shap_values[1], shap_sample.iloc[student_index])
```